

UNIVERSIDADE DE MOGI DAS CRUZES

**POLÍTICA DE SEGURANÇA DA INFORMAÇÃO:
DOCUMENTAÇÃO DOS REQUISITOS DE SEGURANÇA DA INFORMAÇÃO**

MOGI DAS CRUZES

2026

UNIVERSIDADE DE MOGI DAS CRUZES

INTEGRANTES:

GABRIEL CELESTINO PEREZ – RGM: 11231504633;

RAFAEL DE MORAES MANNA – RGM: 11232100493;

RIAN GOMES PEREIRA – RGM: 11232100747;

STEPHANY CANUTO DA SILVA – RGM: 11232101978;

VINICIUS DOS ANJOS NASCIMENTO - RGM: 11232100269.

**DOCUMENTAÇÃO DOS REQUISITOS DE SEGURANÇA DA INFORMAÇÃO NO
SISTEMA SAEC**

MOGI DAS CRUEZES

MAIO/2026

Sumário

1. AUTENTICAÇÃO E GESTÃO DE CREDENCIAIS.....	4
1.1 - Uso de hash criptográfico seguro para senhas (Argon2, bcrypt ou PBKDF2)	4
1.2 - Parâmetros de custo do hash configurados e justificados.....	5
1.3 - Uso de salt criptográfico único por usuário	5
1.4 - Armazenamento correto do hash + salt.....	6
1.5 - Autenticação de dois fatores (2FA) implementada	6
1.6 - Validação do 2FA após autenticação primária.....	7
1.7 - Fluxo de autenticação documentado	7
1.8 Evidências funcionais (prints, logs ou testes).....	9
1.9 - Sessões com tempo de expiração.....	11
1.10 - Invalidação de sessão no logout	11
1.11 - Proteção contra força bruta (rate limit, bloqueio, atraso).....	11
1.12 - Justificativas técnicas documentadas.....	13
2. RECUPERAÇÃO DE SENHA.....	15
2.1 - Funcionalidade de recuperação de senha implementada.....	15
2.2 - Token criptograficamente seguro	17
2.3 - Token com tempo de expiração.....	17
2.4 - Token invalidado após uso	18
2.5 - Falha correta para token expirado.....	18
2.6 - Registro de solicitação de recuperação em log.....	18
2.7 - Registro de sucesso/falha do processo	18
3 - CRIPTOGRAFIA E COMUNICAÇÃO SEGURA	19
3.1 - Comunicação protegida por TLS/HTTPS	19
3.2 - Bloqueio de conexões não seguras.....	19
3.3 - Evidência de tráfego cifrado	19
3.4 - Dados sensíveis criptografados em repouso	20
3.5 - Uso de algoritmo criptográfico adequado.....	21
3.6 - Chaves criptográficas protegidas.....	21
3.7 - Estratégia de criptografia documentada	21
3.8 - Justificativa técnica das escolhas	22
4. - CONFORMIDADE COM A LGPD	22

4.1 - Listagem completa dos dados pessoais coletados	22
4.2 - Associação de cada dado a uma finalidade	22
4.3 - Evidência de minimização de dados.....	23
4.4 - Registro explícito de consentimento.....	23
4.5 - Consentimento associado à finalidade	24
4.6 - Possibilidade de revogação do consentimento.....	25
4.7 - Registro de data e versão do consentimento.....	26
4.8 - Funcionalidade de consulta aos dados do titular	27
4.9 - Funcionalidade de exportação dos dados.....	27
4.10 - Funcionalidade de exclusão dos dados pessoais.....	27
4.11 - Fluxo de atendimento aos direitos documentado	28
4.11.1 - Solicitação do Titular	29
4.11.2 - Validação de Identidade	29
4.11.3 - Registro e Rastreabilidade da Solicitação.....	29
4.11.4 - Análise e Validação da Requisição	29
4.11.5 - Execução da Solicitação	29
4.11.6 - Resposta e Comunicação ao Titular	30
4.11.7 - Medidas de Segurança Aplicadas.....	30
5 - Auditoria e Logs	30
5.1 - Logs de autenticação registrados	30
5.2 - Logs de falhas e 2FA registrados.....	31
5.3 - Proteção contra alteração de LOG	31
5.4 - Exemplo de análise de LOGs apresentados	31
6.1 - Documento de visão geral do sistema	33
6.2 - Diagrama de arquitetura	33
6.3 - Fluxos de autenticação e dados documentados	34
6.4 - Gestão de credenciais documentada	36
6.5 - Uso de criptografia documentado	36
6.6 - Identificação dos ativos do sistema	36
6.7 - Identificação de ameaças e vulnerabilidades	37
6.8 - Associação risco × contramedida	38
6.9 - Registro de logs de tentativa de login.....	38

6.10 - Atualização da matriz risco × contramedida para logs de autenticação.....	39
6.11 - Uso de artigos científicos e/ou normas técnicas.....	40
6.12 - Referências normalizadas	41
7.1 - Resumo entre 200 e 300 palavras.....	41
7.2 - Objetivo Claramente Definido	42
7.3 - Metodologia Técnica Descrita	42
7.4 - Mecanismos de Segurança Apresentados	42
7.5 - Conformidade com a LGPD Explicitada	42
7.6 - Terminologia Técnica Adequada	43
7.7- Qualidade Textual e Científica	43

DOCUMENTAÇÃO DOS REQUISITOS DE SEGURANÇA DA INFORMAÇÃO NO SISTEMA SAEC

1. AUTENTICAÇÃO E GESTÃO DE CREDENCIAIS

1.1 - Uso de hash criptográfico seguro para senhas (Argon2, bcrypt ou PBKDF2)

O projeto utiliza **Argon2** por meio da biblioteca `argon2-cffi`, através da classe `PasswordHasher`. O Argon2 é o algoritmo vencedor da Password Hashing Competition (2015) e é atualmente considerado o padrão mais seguro para hashing de senhas.

```
ph = PasswordHasher(  
    time_cost=3,  
    memory_cost=65536,  
    parallelism=4,  
    hash_len=32,  
    salt_len=8  
)
```

O hash é aplicado no momento do cadastro:

```
def cadastrar_usuario(form):  
  
    conn = None  
    cursor = None  
  
    try:  
  
        if not form.get("termos"):  
  
            return {  
                "success": False,  
                "erro": "Você precisa aceitar os termos."  
            }  
  
        senha = form.get("senha", "").strip()  
        senha_ver = form.get("senhaVER", "").strip()  
  
        if senha != senha_ver:  
  
            return {  
                "success": False,  
                "erro": "As senhas não coincidem"  
            }  
    }
```

```

erro_validacao = validar_nova_senha(senha)

if erro_validacao:

    return {
        "success": False,
        "erro": erro_validacao
    }

if find_user_by_email(form.get("email")):

    return {
        "success": False,
        "erro": "Email já cadastrado"
    }

if find_user_by_cpf(form.get("cpf")):

    return {
        "success": False,
        "erro": "CPF já cadastrado"
    }

if find_user_by_telefone(form.get("telefone")):

    return {
        "success": False,
        "erro": "Telefone já cadastrado"
    }

senha_hash = ph.hash(senha)

```

1.2 - Parâmetros de custo do hash configurados e justificados

- `ph = PasswordHasher( time_cost=3,` - Número de iterações — aumenta o tempo de processamento
- `memory_cost=65536,` - 64 MB de memória RAM exigidos — dificulta ataques com hardware paralelo (GPUs/ASICs)
- `parallelism=4,` - Número de threads paralelas utilizadas
- `hash_len=32,` - Tamanho do hash gerado: 32 bytes (256 bits)
- `salt_len=8` - Tamanho do salt gerado automaticamente: 8 bytes

1.3 - Uso de salt criptográfico único por usuário

O salt é gerado automaticamente pelo PasswordHasher a cada chamada de `ph.hash()`, garantindo que cada usuário receba um salt único e aleatório, conforme o parâmetro:

- `salt_len=8` - **Salt de 8 bytes gerado automaticamente pelo Argon2**
- `senha_hash = ph.hash(form.get("senha"))` - **Salt único gerado internamente a cada chamada**

Isso significa que dois usuários com a mesma senha terão hashes completamente diferentes.

1.4 - Armazenamento correto do hash + salt

O Argon2 armazena o hash e o salt juntos em uma única string codificada, no formato padrão PHC (Password Hashing Competition String Format). Esse valor é salvo diretamente no campo senha do banco de dados: `senha_hash = ph.hash(form.get("senha"))`

```
data = {
    ...
    "senha": senha_hash
    ...
}
```

A string resultante tem formato similar a:

`$argon2id$v=19$m=65536,t=3,p=4$<salt_base64>$<hash_base64>`

Ou seja, nunca é armazenada a senha em texto puro — apenas o hash final com o salt embutido, o que está em conformidade com as boas práticas de segurança.

1.5 - Autenticação de dois fatores (2FA) implementada

```
def gerar_codigo_2fa():

    codigo = secrets.randbelow(900000) + 100000
    expiracao = datetime.now() + timedelta(minutes=5)

    return codigo, expiracao

# =====
# ENVIAR EMAIL 2FA
# =====
def enviar_codigo_email(destinatario, codigo):
```

```

remetente = os.getenv("EMAIL_REMETENTE")
senha = os.getenv("EMAIL_SENHA")

msg = MIMEText(
    f"Seu código de verificação é: {codigo}"
)

msg["Subject"] = "Código de verificação"
msg["From"] = remetente
msg["To"] = destinatario

with smtplib.SMTP("smtp.gmail.com", 587) as servidor:

    servidor.starttls()

    servidor.login(
        remetente,
        senha
    )

    servidor.send_message(msg)

```

1.6 - Validação do 2FA após autenticação primária

O 2FA foi implementado como uma etapa **obrigatória e sequencial** à autenticação primária. Após a validação correta da senha, o sistema **não concede acesso imediato** — em vez disso, os dados do usuário são armazenados em variáveis temporárias de sessão (`usuario_temp_*`) e o usuário é redirecionado para a tela de verificação do código 2FA.

O acesso ao sistema só é liberado após a conclusão bem-sucedida dessa segunda etapa, onde o código é validado contra o valor salvo na sessão, respeitando também seu prazo de expiração. Somente então os dados temporários são promovidos para a sessão autenticada definitiva e o usuário é redirecionado ao sistema.

Esse fluxo garante que **nenhuma sessão autenticada é criada sem a conclusão das duas etapas**, seguindo o princípio de autenticação progressiva e impedindo que a sessão seja estabelecida em caso de comprometimento apenas da senha.

1.7 - Fluxo de autenticação documentado

No desenvolvimento do sistema SAECA, estruturamos o fluxo de autenticação como um mecanismo fundamental para apoiar o controle de acesso às áreas internas da aplicação. Entendemos que a autenticação constitui uma etapa essencial dentro da segurança da informação, pois é por meio dela que o sistema verifica a identidade do usuário antes de conceder acesso a funcionalidades restritas.

O fluxo de autenticação tem início quando o usuário acessa a interface de login e informa suas credenciais, compostas por e-mail e senha. Após o envio dessas informações, o sistema realiza uma consulta ao banco de dados com a finalidade de localizar um registro correspondente ao e-mail informado. Caso não exista um usuário cadastrado com esse identificador, o processo de autenticação é interrompido e o acesso não é autorizado.

Quando o e-mail informado corresponde a um usuário existente, o sistema prossegue para a etapa de validação da senha. Nessa fase, não utilizamos a comparação da senha em texto simples. Em vez disso, adotamos o uso de hash criptográfico, implementado com o algoritmo Argon2, de forma a garantir maior proteção às credenciais armazenadas. Além disso, o algoritmo é empregado com parâmetros de custo configuráveis, o que fortalece o nível de segurança da verificação. Consideramos ainda importante destacar que o uso do Argon2 contempla salt automático, recurso que contribui para dificultar ataques baseados em repetição de hashes e tabelas previamente conhecidas.

Após a validação correta das credenciais, o sistema cria uma sessão temporária para conduzir a etapa complementar de autenticação em dois fatores. Nessa sessão, são armazenadas informações necessárias ao processo, como o identificador temporário do usuário, o código de verificação e o prazo de expiração. Após a confirmação correta do código de 2FA, o sistema registra na sessão informações básicas, como o identificador e o nome do usuário, permitindo que o sistema reconheça quem está logado e mantenha o acesso durante sua navegação. Essa escolha caracteriza o uso de controle de autenticação por sessão, técnica amplamente utilizada em aplicações web por sua eficiência no gerenciamento do estado de autenticação.

Além da validação por e-mail e senha, o código analisado implementa autenticação em dois fatores por e-mail. Nesse processo, o sistema gera um código numérico de seis dígitos, envia esse código ao e-mail do usuário e define um prazo de expiração de cinco minutos. Somente após a confirmação correta desse código o usuário tem a autenticação concluída, o que acrescenta uma camada adicional de segurança ao fluxo de login.

Da mesma forma, implementamos o encerramento da autenticação por meio do logout. Nesse processo, a sessão ativa é invalidada com a limpeza dos dados armazenados, utilizando o comando `session.clear()`. Essa medida corresponde à técnica de invalidação de sessão, sendo importante para evitar que o acesso permaneça ativo após a saída do usuário, especialmente em situações de uso compartilhado do dispositivo.

Embora a sessão seja criada e invalidada corretamente no fluxo de autenticação, a análise do código também indica que as rotas internas ainda não possuem uma verificação obrigatória de sessão aplicada diretamente a cada rota, como um mecanismo do tipo `login_required`. Dessa forma, esse ponto deve ser compreendido como uma melhoria necessária para reforçar o bloqueio efetivo de acesso não autenticado às páginas internas.

Além do fluxo principal de login e logout, adotamos outras práticas complementares que reforçam a segurança do sistema. Entre elas, destacamos o uso de variáveis de ambiente para tratamento de informações sensíveis de configuração, como a conexão com o banco de dados, e a utilização de consultas SQL parametrizadas, que reduzem o risco de injeção de SQL.

Assim, o fluxo de autenticação documentado no SAECA pode ser descrito pelas seguintes etapas: acesso à interface de login, envio das credenciais, busca do usuário no banco de dados, validação segura da senha, envio e validação do código de autenticação em dois fatores, criação da sessão autenticada e encerramento do acesso por meio do logout. Consideramos que esse fluxo atende à necessidade de controle básico de autenticação do sistema e demonstra a adoção de mecanismos iniciais consistentes de proteção das credenciais e da sessão do usuário.

1.8 Evidências funcionais (prints, logs ou testes)

- Cadastro de Usuário:

The screenshot displays a web browser window with the URL `127.0.0.1:5000/cadastrar`. The page title is "Cadastro". The form contains the following fields and elements:

- Nome Completo***: Text input with the value "Gabriel Celestino Perez".
- Email***: Text input with the value "celestinogabriel09@gmail.com".
- CPF***: Text input with the value "441.786.678-31".
- Telefone***: Text input with the value "(11) 97200-2553".
- CEP***: Text input with the value "08531-350".
- Senha***: Password input field with a strength indicator showing four green checkmarks. The indicator text reads: "A senha deve conter: No mínimo 8 caracteres, Uma letra maiúscula, Uma letra minúscula, Um número, Um caractere especial".
- Confirme sua senha***: Password input field for confirmation.
- Buttons**: "Cadastrar" (blue) and "Voltar" (grey).
- Footer**: "Ler termos de uso" and "Aceito os termos*" (checked).

- Resultado:

127.0.0.1:5000/cadastrar

Cadastro

Nome Completo*
Ex: Nome Sobrenome

Email*
endereco@email.com

CPF*
Digite apenas os números do seu CPF

Telefone*
Digite apenas os números do seu telefone

CEP*
Digite apenas os números do seu CEP

[Ler termos de uso](#)
☐ Aceito os termos*

Senha*
Confirme sua senha*
Digite sua senha novamente

Sucesso
Cadastrado com Sucesso :)

✖ Um caractere especial

Cadastrar **Voltar**

- Login/Autenticação em 2FA

127.0.0.1:5000/login

Login

Email
celestinogabriel09@gmail.com

Senha

Login

Não tem login? [Cadastre-se](#)
[Esqueci minha senha](#)
[Voltar](#)

Iniciar

- E-mail de Verificação:

mail.google.com/mail/u/0/#inbox/FMfcgQgLvDXNGmmGyjhCTFSMCHS

Gmail Pesquisar e-mail

Escrever

Caixa de entrada 749

- Com estrela
- Adiados
- Enviados
- Rascunhos
- Compras 33
- Mais

Marcadores +

- Arquivo Morto

Código de verificação Caixa de entrada x

oficialsaeca@gmail.com
para mim

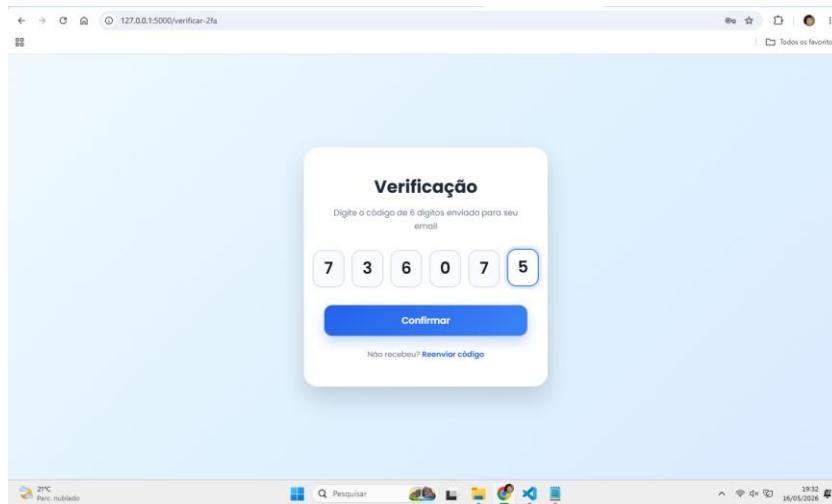
19:30 (há 0 minuto)

Parece que esta mensagem está em inglês
[Traduzir para o português](#)

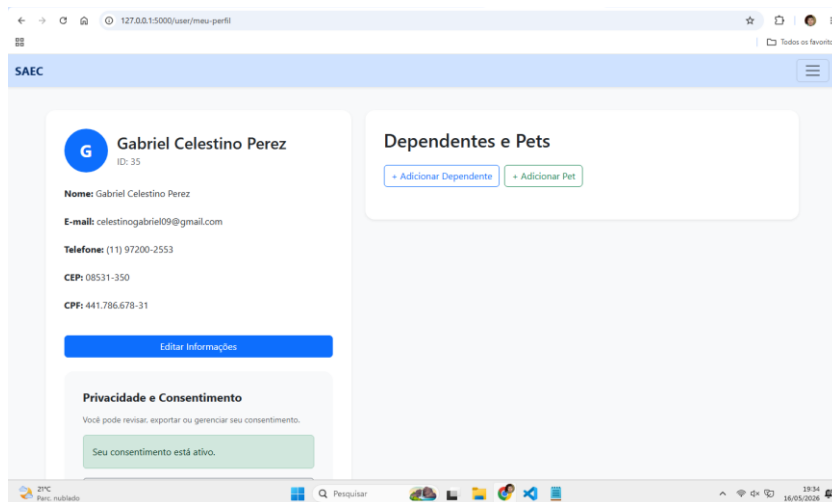
Seu código de verificação é: 736075

Responder **Encaminhar**

- Testando Código de Verificação:



- Resultado:



1.9 - Sessões com tempo de expiração

```
app.permanent_session_lifetime = timedelta(minutes=10)
```

1.10 - Invalidação de sessão no logout

```
@auth_bp.route("/logout")
def logout():
    session.clear()
    return redirect("/")
```

1.11 - Proteção contra força bruta (rate limit, bloqueio, atraso)

```
MAX_TENTATIVAS = 5
```

```

@auth_bp.route("/login", methods=["GET", "POST"])
@limiter.limit("5 per minute")
@limiter.limit("20 per hour")
def login():

    if request.method == "GET":
        session.setdefault("tentativas", 0)
        return render_template("login.html")

    session.setdefault("tentativas", 0)

    # Verifica bloqueio
    if session["tentativas"] >= MAX_TENTATIVAS:

        create_log_auth(
            email=request.form.get("email"),
            sucesso=False,
            ip=request.remote_addr,
            user_agent=request.headers.get("User-Agent"),
            motivo_falha="Conta bloqueada por excesso de tentativas"
        )

        return render_template(
            "login.html",
            tipo="erro",
            tentativas_restantes=0
        )

    result = realizar_login(
        request.form,
        ip=request.remote_addr,
        user_agent=request.headers.get("User-Agent")
    )

    if not result["success"]:
        session["tentativas"] += 1
        tentativas_restantes = MAX_TENTATIVAS - session["tentativas"]

        return render_template(
            "login.html",
            mensagem=result["erro"],
            tipo="erro",
            tentativas_restantes=tentativas_restantes
        )

```

```
usuario = result["usuario"]  
session["tentativas"] = 0
```

1.12 - Justificativas técnicas documentadas

No desenvolvimento do sistema SAECA, procuramos adotar decisões técnicas que contribuíssem para a segurança da aplicação, a organização do projeto e o controle adequado do acesso dos usuários. Nesse contexto, entendemos que a documentação das justificativas técnicas é importante não apenas para registrar as escolhas realizadas, mas também para demonstrar que tais decisões foram tomadas com base em critérios de funcionalidade, proteção das informações e boas práticas de desenvolvimento.

Uma das principais decisões adotadas foi o uso de hash criptográfico para o armazenamento das senhas. Optamos por não manter senhas em texto simples no banco de dados, uma vez que esse tipo de armazenamento representa um risco elevado à segurança dos usuários. Ao registrar apenas a versão protegida da senha, reduzimos o impacto de um eventual comprometimento da base de dados, uma vez que terceiros não teriam acesso imediato às credenciais originais.

Associada a essa escolha, definimos o uso do algoritmo Argon2 como mecanismo de proteção de senhas. Tal decisão se justifica por se tratar de uma solução reconhecida pela sua robustez e adequação a cenários que demandam proteção de credenciais. Além disso, utilizamos parâmetros de custo configuráveis, o que permite tornar o processo de verificação mais resistente. O uso de salt automático, característica inerente ao algoritmo, reforça ainda mais essa proteção, pois evita que senhas idênticas gerem sempre a mesma representação protegida.

Outra decisão técnica identificada no código foi a adoção de autenticação em dois fatores por e-mail. Esse recurso foi incorporado como etapa posterior à validação da senha, exigindo que o usuário informe um código temporário enviado para seu e-mail. A definição de um código de seis dígitos com expiração de cinco minutos contribui para reduzir o risco de acesso indevido apenas com a posse da senha.

Além disso, implementamos mecanismos de proteção contra ataques de força bruta por meio de rate limiting. O sistema utiliza a biblioteca Flask-Limiter para limitar a quantidade de requisições

realizadas pelos usuários, mitigando tentativas automatizadas de acesso indevido e reduzindo significativamente a viabilidade de ataques direcionados às rotas de autenticação.

Foi definido um limite global de requisições aplicável às rotas gerais da aplicação, configurado em até 2000 requisições por dia e 200 requisições por hora por usuário/IP. Para as rotas críticas de autenticação, especialmente relacionadas ao login, foram estabelecidos limites mais restritivos, permitindo no máximo 5 tentativas por minuto e 20 tentativas por hora. Essa abordagem contribui para dificultar ataques automatizados de descoberta de credenciais, além de auxiliar na preservação da disponibilidade e estabilidade da aplicação.

Outra decisão relevante foi a adoção de sessão para manutenção da autenticação. Escolhemos esse mecanismo porque ele permite que o sistema preserve o estado do usuário autenticado durante a navegação, dispensando a necessidade de repetição do login a cada interação. Sob a perspectiva da segurança da informação, essa abordagem favorece a identificação do usuário autenticado e serve como base para o controle do acesso às áreas restritas, ao mesmo tempo em que contribui para a usabilidade da aplicação.

De modo complementar, implementamos a invalidação da sessão no logout, procedimento que encerra adequadamente o vínculo entre o usuário e o sistema ao final da utilização. Consideramos essa medida importante para reduzir riscos de permanência indevida do acesso, sobretudo em ambientes compartilhados ou quando o usuário se afasta do dispositivo após encerrar suas atividades.

Apesar disso, a análise do código também demonstra que a verificação obrigatória da sessão ainda precisa ser aplicada às rotas internas. A implementação de um mecanismo centralizado de proteção, como um decorator `login_required`, permitiria reforçar o controle de acesso às páginas restritas e alinhar de forma mais completa a navegação interna ao fluxo de autenticação já implementado.

Também adotamos o uso de variáveis de ambiente para separar informações sensíveis de configuração do corpo principal do código. Essa decisão favorece uma organização mais segura da aplicação, além de facilitar a administração de parâmetros críticos, como a conexão com o banco de dados. No mesmo sentido, empregamos consultas SQL parametrizadas, técnica

importante para a prevenção de injeção de SQL, uma das vulnerabilidades mais conhecidas em aplicações integradas a bancos de dados.

Além das escolhas diretamente relacionadas à autenticação, organizamos o sistema em camadas distintas, divididas entre rotas, controllers, models e views. Embora essa seja uma decisão arquitetural, entendemos que ela também possui impacto relevante na segurança, pois melhora a estrutura do projeto, facilita processos de manutenção, favorece revisões técnicas e torna mais clara a responsabilidade de cada parte do sistema.

Dessa forma, as justificativas técnicas adotadas por nosso grupo estão fundamentadas em princípios básicos de segurança da informação e de organização de software. Entre as principais decisões, destacamos o uso de hash com Argon2, os parâmetros de custo configuráveis, o uso de salt automático, a autenticação em dois fatores por e-mail, os mecanismos de rate limiting para mitigação de ataques de força bruta, o controle de autenticação por sessão, a invalidação da sessão no logout, o uso de variáveis de ambiente e a adoção de consultas parametrizadas. Consideramos que essas escolhas fornecem uma base adequada para o desenvolvimento de um sistema mais seguro, estruturado e alinhado às exigências de proteção da informação.

2. RECUPERAÇÃO DE SENHA

2.1 - Funcionalidade de recuperação de senha implementada

```
@auth_bp.route("/recuperar-senha", methods=["GET", "POST"])
def recuperar_senha():

    if request.method == "GET":
        return render_template("recuperarSenha.html")

    email = request.form["email"]

    usuario = find_user_by_email(email)

    if not usuario:
        return render_template(
            "recuperarSenha.html",
            mensagem="Email não encontrado",
            tipo="erro"
        )
```

```

codigo, expiracao = gerar_codigo_2fa()

session["email_recuperacao"] = email
session["reset_codigo"]      = codigo
session["reset_expira"]      = expiracao.strftime("%Y-%m-%d %H:%M:%S")

enviar_codigo_email(email, codigo)
current_app.logger.debug("Codigo recuperacao: %s", codigo)

return redirect(url_for("auth.validar_codigo_recuperacao"))

@auth_bp.route("/validar-recuperacao", methods=["GET", "POST"])
def validar_codigo_recuperacao():

    if request.method == "GET":
        return render_template("validarCodigoRecuperacao.html")

    expiracao_datetime = datetime.strptime(expiracao, "%Y-%m-%d %H:%M:%S")

    if datetime.now() > expiracao_datetime:

        session.pop("reset_codigo", None)
        session.pop("reset_expira", None)
        session.pop("email_recuperacao", None)

        return render_template(
            "validarCodigoRecuperacao.html",
            mensagem="Código expirado. Solicite outro código.",
            tipo="erro"
        )

    session["reset_validado"] = True

    session.pop("reset_codigo", None)
    session.pop("reset_expira", None)

    return redirect(url_for("auth.nova_senha"))

@auth_bp.route("/nova-senha", methods=["GET", "POST"])
def nova_senha():

```

```

email          = session.get("email_recuperacao")
reset_validado = session.get("reset_validado")

if not email or not reset_validado:
    return redirect(url_for("auth.recuperar_senha"))

if request.method == "GET":
    return render_template("novaSenha.html")

elif request.method == "POST":

    resultado = redefinir_senha(
        email=email,
        form=request.form
    )

    if not resultado["success"]:
        return render_template(
            "novaSenha.html",
            mensagem=resultado["erro"],
            tipo="erro"
        )

    session.pop("email_recuperacao", None)
    session.pop("reset_validado", None)

    return redirect(url_for("auth.login"))

return render_template("novaSenha.html")

```

2.2 - Token criptograficamente seguro

```

def gerar_codigo_2fa():

    codigo = secrets.randbelow(900000) + 100000

    expiracao = datetime.now() + timedelta(minutes=5)

    return codigo, expiracao

```

2.3 - Token com tempo de expiração

```

codigo_digitado = request.form["codigo"]
codigo_salvo    = session.get("reset_codigo")
expiracao       = session.get("reset_expira")

if not codigo_salvo or not expiracao:

    session.clear()

    return render_template(
        "validarCodigoRecuperacao.html",
        mensagem="Sessão expirada. Solicite um novo código.",
        tipo="erro"
    )

```

2.4 - Token invalidado após uso

```

if codigo_digitado != codigo_salvo:

    return render_template(
        "validarCodigoRecuperacao.html",
        mensagem="Código inválido.",
        tipo="erro"
    )

```

2.5 - Falha correta para token expirado

```

[2026-05-30 18:33:52,575] DEBUG in auth: Codigo 2FA: 747935
127.0.0.1 - - [30/May/2026 18:33:56] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [30/May/2026 18:33:56] "GET /verificar-2fa HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 18:33:56] "GET /static/css/2fa.css HTTP/1.1" 304 -
127.0.0.1 - - [30/May/2026 18:41:31] "POST /verificar-2fa HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 18:41:31] "GET /static/css/2fa.css HTTP/1.1" 304 -

```

2.6 - Registro de solicitação de recuperação em log

```

127.0.0.1 - - [03/Jun/2026 22:42:46] "POST /recuperar-senha HTTP/1.1" 302 -
127.0.0.1 - - [03/Jun/2026 22:42:46] "GET /validar-recuperacao HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:42:46] "GET /static/css/verificarCodigoStyle.css HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:43:35] "POST /validar-recuperacao HTTP/1.1" 302 -
127.0.0.1 - - [03/Jun/2026 22:43:35] "GET /nova-senha HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:43:35] "GET /static/assets/login.jpg HTTP/1.1" 304 -
127.0.0.1 - - [03/Jun/2026 22:43:35] "GET /static/css/novaSenha.css HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:43:35] "GET /static/js/validacaoSenha.js HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:43:52] "POST /nova-senha HTTP/1.1" 302 -
127.0.0.1 - - [03/Jun/2026 22:43:52] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:43:52] "GET /static/assets/login.jpg HTTP/1.1" 304 -
127.0.0.1 - - [03/Jun/2026 22:43:52] "GET /static/css/loginCadastroStyle.css HTTP/1.1" 200 -

```

2.7 - Registro de sucesso/falha do processo

Sucesso:

```
127.0.0.1 - - [03/Jun/2026 22:51:36] "GET /static/css/verificarCodigoStyle.css HTTP/1.1" 304 -
127.0.0.1 - - [03/Jun/2026 22:51:50] "POST /validar-recuperacao HTTP/1.1" 302 -
127.0.0.1 - - [03/Jun/2026 22:51:50] "GET /nova-senha HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:51:50] "GET /static/css/novaSenha.css HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:51:50] "GET /static/assets/login.jpg HTTP/1.1" 304 -
127.0.0.1 - - [03/Jun/2026 22:51:50] "GET /static/js/validacaoSenha.js HTTP/1.1" 304 -
127.0.0.1 - - [03/Jun/2026 22:52:06] "POST /nova-senha HTTP/1.1" 302 -
127.0.0.1 - - [03/Jun/2026 22:52:06] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:52:07] "GET /static/assets/login.jpg HTTP/1.1" 304 -
127.0.0.1 - - [03/Jun/2026 22:52:07] "GET /static/css/loginCadastroStyle.css HTTP/1.1" 200 -
```

Falha:

```
127.0.0.1 - - [03/Jun/2026 22:49:19] "POST /recuperar-senha HTTP/1.1" 302 -
127.0.0.1 - - [03/Jun/2026 22:49:19] "GET /validar-recuperacao HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:49:19] "GET /static/css/verificarCodigoStyle.css HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:49:25] "POST /validar-recuperacao HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:49:25] "GET /static/css/verificarCodigoStyle.css HTTP/1.1" 304 -
127.0.0.1 - - [03/Jun/2026 22:49:29] "POST /validar-recuperacao HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:49:29] "GET /static/css/verificarCodigoStyle.css HTTP/1.1" 304 -
```

3 - CRIPTOGRAFIA E COMUNICAÇÃO SEGURA

3.1 - Comunicação protegida por TLS/HTTPS

O Flask-Talisman foi configurado para forçar HTTPS em todas as conexões, e os cookies de sessão foram configurados para trafegar exclusivamente por conexões seguras:

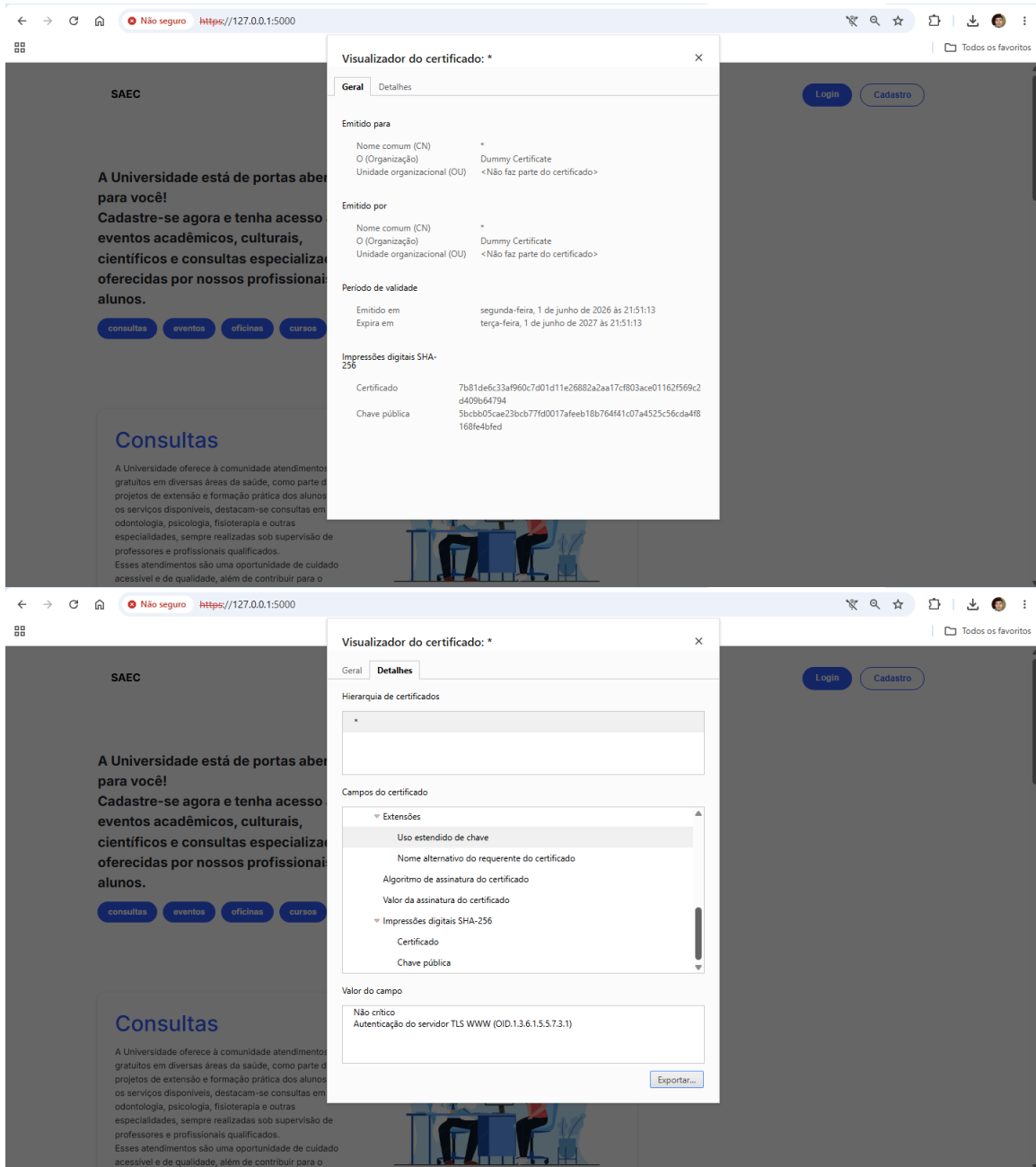
```
app.config['SESSION_COOKIE_SECURE'] = True
```

3.2 - Bloqueio de conexões não seguras

O parâmetro `force_https=True` do Talisman garante que qualquer requisição HTTP é automaticamente recusada e redirecionada para HTTPS, bloqueando conexões não cifradas:

```
Talisman(app, force_https=True, content_security_policy=None)
```

3.3 - Evidência de tráfego cifrado



3.4 - Dados sensíveis criptografados em repouso

A senha dos usuários nunca é armazenada em texto puro no banco de dados. Antes de qualquer gravação na tabela clientes, ela é transformada em um hash criptográfico irreversível:

#

app/controllers/user_controller.py

```
senha_hash = ph.hash(senha)
# apenas o hash é salvo — tanto no cadastro quanto na redefinição de senha
```

O valor armazenado na coluna senha segue o formato \$argon2id\$v=19\$m=65536,..., completamente ilegível, impossibilitando a recuperação da senha original mesmo com acesso direto ao banco.

3.5 - Uso de algoritmo criptográfico adequado

O sistema utiliza o algoritmo **Argon2id** (biblioteca `argon2-cffi`), vencedor da Password Hashing Competition (PHC) e recomendado pela OWASP/NIST para proteção de senhas. A instância é configurada em `user_controller.py`:

```
ph = PasswordHasher(
    time_cost=3,          - número de iterações
    memory_cost=65536,    - 64 MB de RAM por operação
    parallelism=4,        - threads paralelas
    hash_len=32,          - hash de 256 bits
    salt_len=8            - salt único e aleatório por hash
)
```

O salt gerado automaticamente garante que senhas iguais produzam hashes diferentes, prevenindo ataques de rainbow table. Na autenticação, a verificação é feita com `ph.verify(hash_banco, senha_digitada)`, método timing-safe que nunca expõe o valor original.

3.6 - Chaves criptográficas protegidas

A chave secreta da aplicação é gerada dinamicamente a cada inicialização usando `secrets.token_hex(32)`, que produz 32 bytes (256 bits) de entropia criptograficamente segura. Ela nunca é hardcoded no código-fonte:

```
app.secret_key = secrets.token_hex(32)
```

3.7 - Estratégia de criptografia documentada

A estratégia adotada combina TLS para proteção em trânsito, cookies seguros para proteção da sessão, e geração dinâmica de chaves para proteção da aplicação:

```
app.config['SESSION_COOKIE_SECURE'] = True      # Cookie só trafega em HTTPS
app.config['SESSION_COOKIE_HTTPONLY'] = True    # Bloqueia acesso via JavaScript (anti-XSS)
```

```
app.config['SESSION_COOKIE_SAMESITE'] = 'Lax' # Restringe envio cross-site (anti-CSRF)
Talisman(app, force_https=True, content_security_policy=None)
```

3.8 - Justificativa técnica das escolhas

Escolha:	Justificativa:
SESSION_COOKIE_SECURE	Impede que o cookie vaze em conexões HTTP simples
SESSION_COOKIE_HTTPONLY	Previne roubo de sessão via XSS
SESSION_COOKIE_SAMESITE=Lax	Mitiga CSRF sem quebrar navegação normal
force_https=True	Garante que nenhuma requisição trafega sem criptografia
secrets.token_hex(32)	Gera chave com 256 bits de entropia, seguindo recomendações do NIST

4. - CONFORMIDADE COM A LGPD

4.1 - Listagem completa dos dados pessoais coletados

Dados Coletados		
Usuário Comum	Usuário Menor de Idade	PET
Nome Completo	Nome Completo	Nome Completo
CPF	CPF	SinPatinhas
Data de Nascimento	Data de Nascimento	Cor
E-mail	E-mail do Responsável	E-mail do Responsável
Telefone	Telefone do Responsável	Raça
-----	CPF do Responsável	CPF do Responsável

4.2 - Associação de cada dado a uma finalidade

Categoria	Dados Utilizados	Finalidade
Identificação do usuário	Nome completo e CPF	Identificar de forma única o usuário no sistema, evitando duplicidade de cadastro e garantindo a integridade das informações
Gestão de agendamentos	Nome completo e CPF	Criar, alterar, cancelar e vincular agendamentos ao usuário correto
Comprovação de Maioridade	Data de Nascimento	A data de nascimento será utilizada para verificação de faixa etária do usuário. Caso seja identificado que o usuário possui menos de 18 anos, o sistema exigirá a vinculação do

		cadastro a um responsável legal maior de idade, garantindo conformidade com requisitos legais e maior segurança no uso da plataforma.
Comunicação	E-mail e telefone	Enviar confirmações, lembretes, notificações de alterações e prestar suporte
Identificação do PET	Nome do pet, raça, cor, SinPatinhas	Identificar o animal, personalizar o atendimento e manter histórico
Responsável legal (menor)	Nome, CPF do responsável	Garantir autorização legal, vincular menor ao responsável e permitir contato
Segurança e prevenção de fraudes	CPF	Evitar cadastros falsos, validar identidade e prevenir uso indevido

4.3 - Evidência de minimização de dados

Medidas de Minimização Aplicadas

- O sistema solicita apenas dados estritamente necessários para utilização da plataforma.
- Campos opcionais são identificados claramente ao usuário.
- Dados não essenciais não são coletados.
- O sistema evita armazenamento excessivo de informações pessoais.
- Informações sensíveis não são coletadas sem necessidade específica e justificativa legal.
- Os dados possuem finalidade previamente definida e documentada.
- O acesso aos dados é restrito conforme perfil de usuário e necessidade operacional.

Redução de Exposição de Dados

- Para reduzir riscos de vazamento e exposição indevida:
- senhas são armazenadas utilizando hash criptográfico;
- logs evitam exibir informações sensíveis;
- acessos administrativos são controlados;
- dados antigos ou desnecessários podem ser removidos conforme política de retenção.

Conformidade com a LGPD

A minimização de dados aplicada no sistema segue os princípios da necessidade, finalidade e adequação previstos na Lei Geral de Proteção de Dados Pessoais, garantindo que apenas informações indispensáveis sejam tratadas pela aplicação.

4.4 - Registro explícito de consentimento

Durante o processo de cadastro no sistema, será apresentado ao usuário um Termo de Consentimento contendo a descrição clara dos dados coletados e suas respectivas finalidades. O aceite será solicitado de forma explícita, por meio de uma ação afirmativa do usuário (checkbox), garantindo que o consentimento seja livre, informado e inequívoco.

Para fins de conformidade com a LGPD e rastreabilidade, o sistema realizará o registro do consentimento, armazenando as seguintes informações:

- Data e hora do aceite
- Endereço IP e/ou identificação do dispositivo utilizado
- Versão do termo de consentimento aceito
- Identificação do usuário

Esses registros têm como finalidade comprovar a obtenção do consentimento, possibilitar auditorias e garantir a conformidade legal no tratamento de dados pessoais.

4.5 - Consentimento associado à finalidade

1. Dados Coletados

Poderão ser coletados os seguintes **dados pessoais**:

- Nome completo
- CPF
- Data de Nascimento
- E-mail
- Telefone

No caso de menores de idade:

- Nome do responsável legal
- CPF do responsável legal

No caso de cadastro de PET:

- Nome do animal
- Registro (SinPatinhas)
- Raça
- Cor

2. Finalidade do Tratamento

Os dados pessoais coletados serão utilizados para:

- Identificação do usuário no sistema

- Realização, gestão e organização de agendamentos
- Comunicação (confirmações, lembretes e suporte)
- Prestação adequada dos serviços solicitados
- Identificação e atendimento de animais (PETs)
- Garantia de segurança e prevenção de fraudes
- Cumprimento de obrigações legais e regulatórias
- Melhoria contínua dos serviços oferecidos

3. Compartilhamento de Dados

- Os dados poderão ser compartilhados apenas quando necessário, com:
- Profissionais responsáveis pelo atendimento
- Sistemas e serviços utilizados para operação da plataforma
- Autoridades legais, quando exigido por lei

4. Armazenamento e Segurança

Os dados serão armazenados em ambiente seguro, com adoção de medidas técnicas e administrativas adequadas para proteção contra acessos não autorizados, vazamentos ou qualquer forma de tratamento inadequado.

5. Direitos do Titular dos Dados

O(a) usuário(a) poderá, a qualquer momento:

- Solicitar acesso aos seus dados
- Corrigir dados incompletos ou desatualizados
- Solicitar a exclusão dos dados, quando aplicável
- Revogar o consentimento

6. Consentimento

Ao marcar a opção de aceite, o(a) usuário(a) declara que:

Leu e compreendeu este Termo

Concorda de forma livre, informada e inequívoca com o tratamento de seus dados pessoais

No caso de menor de idade, o responsável legal declara que autoriza o tratamento dos dados do menor.

4.6 - Possibilidade de revogação do consentimento

```
def revogar_consentimento(cliente_id):
```

```

conn = get_connection()
cursor = conn.cursor()

try:

    cursor.execute("""
        UPDATE consentimentos_termos
        SET
            aceitou = FALSE,
            data_revogacao = CURRENT_TIMESTAMP
        WHERE cliente_id = %s
            AND aceitou = TRUE
            AND data_revogacao IS NULL
    """, (cliente_id,))

    conn.commit()

finally:
    cursor.close()
    conn.close()

```

4.7 - Registro de data e versão do consentimento

```

def salvar_consentimento(
    cursor,
    cliente_id,
    aceitou=True,
    versao_termo="1.0",
    ip_aceite=None,
    user_agent=None
):

    cursor.execute("""
        INSERT INTO consentimentos_termos (
            cliente_id,
            aceitou,
            versao_termo,
            ip_aceite,
            user_agent
        )
        VALUES (%s, %s, %s, %s, %s)
    """, (
        cliente_id,
        aceitou,
        versao_termo,

```

```

        ip_aceite,
        user_agent
    ))

```

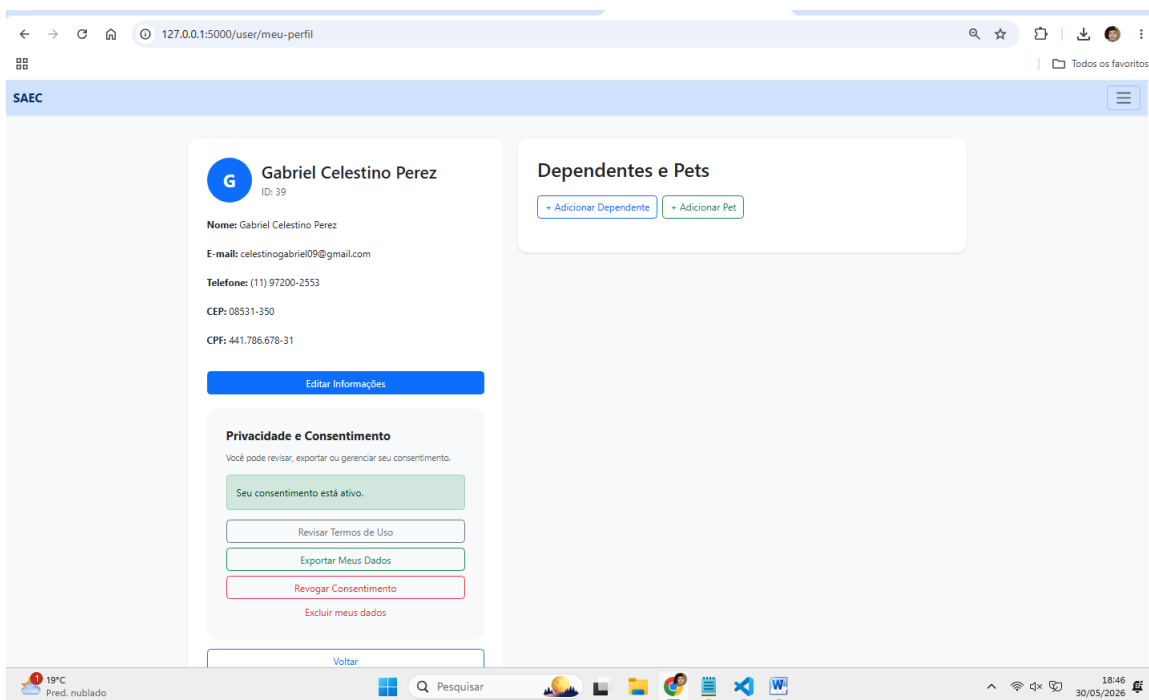
LOG:

```

127.0.0.1 - - [30/May/2026 18:43:37] "GET /user/meu-perfil HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 18:43:37] "GET /static/css/userStyle.css HTTP/1.1" 304 -
127.0.0.1 - - [30/May/2026 18:47:31] "POST /revogar-consentimento HTTP/1.1" 302 -
127.0.0.1 - - [30/May/2026 18:47:31] "GET /user/meu-perfil HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 18:47:31] "GET /static/css/userStyle.css HTTP/1.1" 304 -
127.0.0.1 - - [30/May/2026 18:47:33] "POST /aceitar-termos HTTP/1.1" 302 -
127.0.0.1 - - [30/May/2026 18:47:34] "GET /user/meu-perfil HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 18:47:34] "GET /static/css/userStyle.css HTTP/1.1" 304 -

```

4.8 - Funcionalidade de consulta aos dados do titular



4.9 - Funcionalidade de exportação dos dados

```

@main_bp.route("/user/exportar-pdf")
@login_required
def exportar_pdf():
    return exportar_pdf_controller()

```

4.10 - Funcionalidade de exclusão dos dados pessoais

```

def excluir_dados_banco(cliente_id):

    conn = get_connection()
    cursor = conn.cursor()

```

```
try:

    cursor.execute("""
        DELETE FROM dependentes
        WHERE cliente_id = %s
    """, (cliente_id,))

    cursor.execute("""
        DELETE FROM pets
        WHERE cliente_id = %s
    """, (cliente_id,))

    cursor.execute("""
        DELETE FROM consentimentos_termos
        WHERE cliente_id = %s
    """, (cliente_id,))

    cursor.execute("""
        DELETE FROM enderecos
        WHERE cliente_id = %s
    """, (cliente_id,))

    cursor.execute("""
        DELETE FROM clientes
        WHERE id = %s
    """, (cliente_id,))

    conn.commit()

    return True

except Exception as e:

    conn.rollback()
    print(f"Erro ao excluir cliente: {e}")
    return False

finally:
    cursor.close()
    conn.close()
```

4.11 - Fluxo de atendimento aos direitos documentado

4.11.1 - Solicitação do Titular

O titular dos dados poderá exercer seus direitos por meio dos canais oficiais disponibilizados pela plataforma, como painel do usuário, formulário eletrônico ou canal de suporte. As solicitações poderão envolver consulta, correção, exclusão, anonimização, exportação de dados ou revogação de consentimento.

4.11.2 - Validação de Identidade

Antes do processamento da solicitação, o sistema realizará mecanismos de validação de identidade para garantir que apenas o titular legítimo ou responsável autorizado possa acessar ou solicitar alterações relacionadas aos dados pessoais, reduzindo riscos de fraude, acesso indevido ou engenharia social.

4.11.3 - Registro e Rastreabilidade da Solicitação

Todas as solicitações realizadas serão registradas em logs de auditoria, garantindo rastreabilidade e controle operacional. Os registros poderão conter:

- identificador do usuário;
- tipo de solicitação realizada;
- data e horário da requisição;
- responsável pelo atendimento;
- status do processamento;
- data de conclusão da solicitação.

4.11.4 - Análise e Validação da Requisição

A solicitação passará por análise técnica e administrativa para validação de legitimidade, verificação de conformidade legal e identificação de possíveis obrigações regulatórias ou retenções obrigatórias que possam impedir alteração ou exclusão imediata das informações.

4.11.5 - Execução da Solicitação

Após validação e aprovação da requisição, o sistema poderá executar as seguintes ações:

disponibilização dos dados pessoais ao titular;

- correção de informações cadastrais;
- exportação de dados;
- anonimização;
- exclusão de dados pessoais;
- revogação do consentimento;
- limitação ou interrupção do tratamento de dados, quando aplicável.

4.11.6 - Resposta e Comunicação ao Titular

Após o processamento da solicitação, o titular receberá confirmação formal contendo o resultado da requisição, respeitando os prazos definidos pela política interna da organização e pela Lei Geral de Proteção de Dados Pessoais.

4.11.7 - Medidas de Segurança Aplicadas

O fluxo de atendimento implementa controles de segurança para proteção dos dados pessoais durante todo o processo, incluindo:

- controle de acesso baseado em privilégios;
- autenticação de usuários administrativos;
- registro de logs de auditoria;
- proteção contra alterações indevidas;
- monitoramento das operações realizadas sobre dados pessoais;
- rastreabilidade das ações executadas pela equipe responsável.

4.11.8 - Evidência de Conformidade

O fluxo documentado garante conformidade com os princípios de finalidade, necessidade, transparência, segurança e livre acesso previstos na Lei Geral de Proteção de Dados Pessoais, assegurando tratamento adequado e governança sobre os dados pessoais armazenados pela aplicação.

5 - Auditoria e Logs

5.1 - Logs de autenticação registrados

```
[2026-05-30 17:56:36,886] DEBUG in auth:Codigo 2FA: 648934
127.0.0.1 - - [30/May/2026 17:56:37] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [30/May/2026 17:56:40] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [30/May/2026 17:56:40] "GET /verificar-2fa HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:56:40] "GET /static/css/2fa.css HTTP/1.1" 304 -
127.0.0.1 - - [30/May/2026 17:59:19] "POST /verificar-2fa HTTP/1.1" 302 -
127.0.0.1 - - [30/May/2026 17:59:20] "GET /user/meu-perfil HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:59:20] "GET /static/css/userStyle.css HTTP/1.1" 200 -
```

5.2 - Logs de falhas e 2FA registrados

```
[2026-05-30 18:03:04,851] DEBUG in auth:Codigo 2FA: 594326
127.0.0.1 - - [30/May/2026 18:03:09] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [30/May/2026 18:03:09] "GET /verificar-2fa HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 18:03:09] "GET /static/css/2fa.css HTTP/1.1" 304 -
127.0.0.1 - - [30/May/2026 18:03:38] "POST /verificar-2fa HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 18:03:38] "GET /static/css/2fa.css HTTP/1.1" 304 -
```

5.3 - Proteção contra alteração de LOG

A imutabilidade dos registros de log é garantida por meio de políticas de Row Level Security (RLS) aplicadas diretamente no banco de dados Supabase. Essa abordagem assegura que, mesmo que um usuário autenticado ou processo da aplicação tente modificar ou excluir um registro de log, o próprio banco de dados bloqueará a operação em nível de linha.

As políticas RLS foram configuradas para permitir exclusivamente a operação de inserção (INSERT) nas tabelas de log, removendo qualquer permissão de UPDATE ou DELETE. Dessa forma, a proteção não depende da camada de aplicação, ela é imposta pelo banco de dados, sendo transparente e independente do código Flask.

Essa estratégia garante que:

- Nenhum registro de log pode ser apagado, nem pelo sistema, nem por um eventual atacante que comprometa a aplicação
- Nenhum registro pode ser retroativamente alterado, preservando a integridade das evidências de auditoria
- A proteção opera em nível de infraestrutura, não podendo ser contornada por mudanças no código da aplicação
-

5.4 - Exemplo de análise de LOGs apresentados

Recuperação de senha (22:42–22:43):

```

127.0.0.1 - - [03/Jun/2026 22:42:46] "POST /recuperar-senha HTTP/1.1" 302 -
127.0.0.1 - - [03/Jun/2026 22:42:46] "GET /validar-recuperacao HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:42:46] "GET /static/css/verificarCodigoStyle.css HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:43:35] "POST /validar-recuperacao HTTP/1.1" 302 -
127.0.0.1 - - [03/Jun/2026 22:43:35] "GET /nova-senha HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:43:35] "GET /static/assets/login.jpg HTTP/1.1" 304 -
127.0.0.1 - - [03/Jun/2026 22:43:35] "GET /static/css/novaSenha.css HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:43:35] "GET /static/js/validacaoSenha.js HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:43:52] "POST /nova-senha HTTP/1.1" 302 -
127.0.0.1 - - [03/Jun/2026 22:43:52] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [03/Jun/2026 22:43:52] "GET /static/assets/login.jpg HTTP/1.1" 304 -
127.0.0.1 - - [03/Jun/2026 22:43:52] "GET /static/css/loginCadastroStyle.css HTTP/1.1" 200 -

```

- POST /recuperar-senha → redirecionamento (302), indicando envio do formulário
- GET /validar-recuperacao → sucesso (200), token de recuperação validado
- POST /validar-recuperacao → redirecionamento (302), confirmação processada
- GET /nova-senha → sucesso (200), página de nova senha exibida
- POST /nova-senha → redirecionamento (302), nova senha submetida com sucesso

Autenticação bem-sucedida (30/Mai/2026, 17:56):

- Código 2FA gerado e registrado em DEBUG: 648934
- POST /login → 302, credenciais validadas
- GET /verificar-2fa → 200, página do segundo fator exibida
- POST /verificar-2fa → 302, código 2FA confirmado
- GET /user/meu-perfil → 200, acesso concedido ao perfil do usuário

```

[2026-05-30 17:56:36,886] DEBUG in auth:Codigo 2FA: 648934
127.0.0.1 - - [30/May/2026 17:56:37] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [30/May/2026 17:56:40] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [30/May/2026 17:56:40] "GET /verificar-2fa HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:56:40] "GET /static/css/2fa.css HTTP/1.1" 304 -
127.0.0.1 - - [30/May/2026 17:59:19] "POST /verificar-2fa HTTP/1.1" 302 -
127.0.0.1 - - [30/May/2026 17:59:20] "GET /user/meu-perfil HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 17:59:20] "GET /static/css/userStyle.css HTTP/1.1" 200 -

```

Tentativa com falha no 2FA (30/Mai/2026, 18:03):

- Código 2FA gerado: 594326
- POST /login → 302, credenciais aceitas normalmente
- POST /verificar-2fa → 302, código submetido

- Não há redirecionamento para /user/meu-perfil — o fluxo retorna ao /verificar-2fa com CSS recarregado (304)
- Indica código 2FA incorreto ou expirado, mantendo o usuário bloqueado na etapa de verificação

```
[2026-05-30 18:03:04,851] DEBUG in auth:Codigo 2FA: 594326
127.0.0.1 - - [30/May/2026 18:03:09] "POST /login HTTP/1.1" 302 -
127.0.0.1 - - [30/May/2026 18:03:09] "GET /verificar-2fa HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 18:03:09] "GET /static/css/2fa.css HTTP/1.1" 304 -
127.0.0.1 - - [30/May/2026 18:03:38] "POST /verificar-2fa HTTP/1.1" 200 -
127.0.0.1 - - [30/May/2026 18:03:38] "GET /static/css/2fa.css HTTP/1.1" 304 -
```

6.1 - Documento de visão geral do sistema

O sistema SAECA consiste em um protótipo de aplicação web voltado ao cadastro, autenticação e gerenciamento de informações de usuários, dependentes e pets, com incorporação de práticas de Segurança da Informação e proteção de dados pessoais. A solução foi concebida para demonstrar, em ambiente acadêmico e aplicado, como controles técnicos podem ser integrados a funcionalidades comuns de sistemas web.

A aplicação utiliza Python com o framework Flask no backend, SupaBase para persistência dos dados e tecnologias de interface como HTML, CSS, JavaScript e Bootstrap. A organização do projeto segue uma divisão em camadas, contemplando rotas, controladores, modelos, utilitários e templates de apresentação.

A versão atualizada do sistema contempla cadastro de usuário, autenticação com verificação em duas etapas, recuperação e redefinição de senha, gerenciamento de perfil, cadastro de dependentes e pets, registro e revogação de consentimento, exportação dos dados em PDF e exclusão dos dados do usuário. Essa última funcionalidade reforça o atendimento ao direito de eliminação de dados pessoais pelo titular.

As principais funcionalidades documentadas são:

- cadastro de usuários com dados pessoais e endereço;
- cadastro e gerenciamento de dependentes e pets;
- autenticação por e-mail e senha com validação por Argon2;
- autenticação em dois fatores por código temporário enviado por e-mail;
- recuperação de senha com redefinição e gravação segura do novo hash;
- gerenciamento de perfil e atualização de dados cadastrais;
- registro, consulta, revogação e novo aceite de consentimento;
- exportação dos dados pessoais em PDF;
- exclusão dos dados pessoais e encerramento da sessão após a operação.

6.2 - Diagrama de arquitetura

A arquitetura do SAECA pode ser compreendida como uma aplicação web em camadas. O usuário interage com a interface visual, as requisições são recebidas pelas rotas Flask, as regras de negócio são processadas por controladores, os modelos acessam o banco de dados e serviços auxiliares executam tarefas específicas, como envio de e-mail, geração de PDF, hash de senhas e limitação de tentativas de autenticação.

Figura 1 – Diagrama de arquitetura em camadas do Sistema SAECA.

Camada / componente	Responsabilidade técnica
Interface web	Apresenta formulários, telas de login, cadastro, perfil, recuperação de senha, consentimento e exclusão de dados.
Rotas Flask	Recebem requisições HTTP e direcionam os fluxos de autenticação, cadastro, perfil, consentimento e exclusão.
Controladores	Executam validações, regras de negócio e orquestram chamadas aos modelos.
Modelos	Realizam consultas parametrizadas, inserções, atualizações e exclusões no SupaBase.
SupaBase	Armazena clientes, endereços, dependentes, pets, consentimentos e credenciais protegidas.
Serviços auxiliares	Incluem SMTP para e-mail, Argon2 para hash, Flask-Limiter para limitação de tentativas, ReportLab para PDF e dotenv para variáveis de ambiente.

6.3 - Fluxos de autenticação e dados documentados

Os fluxos documentados descrevem o ciclo de tratamento de dados desde o cadastro até o gerenciamento posterior pelo titular. Considera-se que as correções apontadas pela equipe foram incorporadas ao projeto, incluindo a redefinição segura de senha, o tratamento do arquivo .env fora do versionamento e o encerramento da sessão após exclusão de dados.

Figura 2 – Fluxo de cadastro e tratamento de dados.



Figura 3 – Fluxo de autenticação e recuperação de senha.

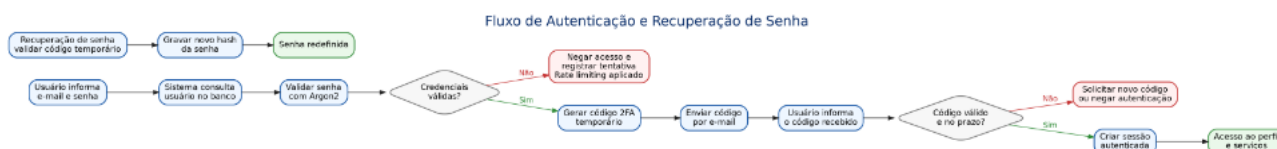
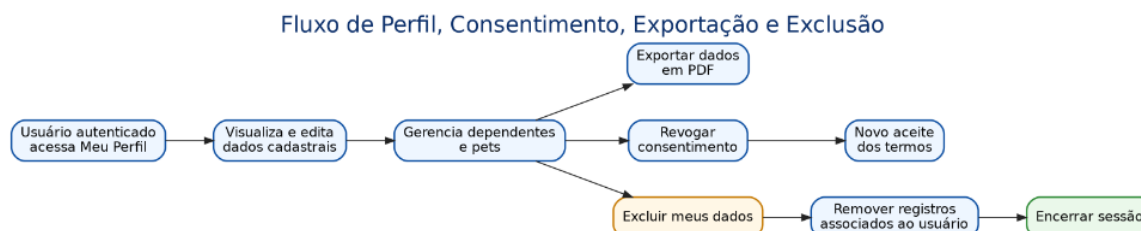


Figura 4 – Fluxo de perfil, consentimento, exportação e exclusão.



No fluxo de cadastro, o usuário informa seus dados pessoais, endereço e, quando aplicável, dados de dependentes e pets. Antes da persistência, o sistema valida os campos, exige aceite dos termos de consentimento e verifica duplicidade de e-mail, CPF e telefone. Após as validações, a senha é protegida com hash Argon2 e os dados são armazenados no Supabase, juntamente com o registro do consentimento.

No fluxo de autenticação, o usuário informa e-mail e senha. O sistema consulta o banco, verifica a senha contra o hash armazenado e, se as credenciais forem válidas, gera um código temporário de autenticação em dois fatores. O código é enviado por e-mail e precisa ser validado dentro do prazo para que a sessão autenticada seja criada.

No fluxo de recuperação de senha, o usuário solicita a recuperação, recebe um código temporário por e-mail, valida esse código e define uma nova senha. A nova senha é armazenada somente após o processo de hash, mantendo o padrão de proteção aplicado às credenciais do sistema.

No fluxo de perfil, o usuário autenticado pode visualizar e editar dados cadastrais, gerenciar dependentes e pets, consultar ou revogar consentimento, aceitar novamente os termos, exportar seus dados em PDF e solicitar a exclusão de seus dados. Após a exclusão, os registros associados ao usuário são removidos e a sessão é encerrada para impedir acessos posteriores com o mesmo estado autenticado.

Fluxo	Etapas principais	Resultado esperado
Cadastro	Validação dos dados, aceite dos termos, verificação de duplicidade, hash da senha, persistência dos dados e registro do consentimento.	Usuário cadastrado com credencial protegida e consentimento registrado.
Autenticação	Consulta ao banco, validação da senha, geração de código 2FA, envio por e-mail e criação de sessão.	Acesso concedido somente após validação de credenciais e segundo fator.
Recuperação de senha	Solicitação, envio de código,	Senha redefinida com proteção

	validação do código, definição de nova senha e gravação do novo hash.	equivalente à do cadastro.
Perfil e consentimento	Edição de dados, exportação em PDF, revogação, novo aceite e consulta de status.	Usuário exerce controle sobre seus dados e consentimento.
Exclusão de dados	Confirmação da operação, remoção dos registros associados e encerramento da sessão.	Dados pessoais eliminados e acesso autenticado encerrado.

6.4 - Gestão de credenciais documentada

A gestão de credenciais contempla as senhas dos usuários e as credenciais técnicas necessárias ao funcionamento da aplicação. As senhas são protegidas por Argon2, e a autenticação ocorre por comparação entre a senha informada e o hash armazenado no banco de dados. A senha original não é mantida em texto puro.

As credenciais técnicas da aplicação são tratadas por variáveis de ambiente, tais como DATABASE_URL, EMAIL_REMETENTE e EMAIL_SENHA. O arquivo .env utilizado para armazenar valores reais permanece fora do versionamento por meio do .gitignore, o que reduz o risco de exposição acidental de dados sensíveis no repositório.

Essa prática permite separar configuração sensível do código-fonte e facilita a adoção de diferentes credenciais para ambientes de desenvolvimento, teste e produção.

6.5 - Uso de criptografia documentado

O principal uso criptográfico do sistema está no armazenamento seguro de senhas por meio do algoritmo Argon2. Durante o cadastro e a redefinição de senha, o valor informado pelo usuário é transformado em um hash antes de ser gravado no banco. Durante o login, a senha digitada é verificada contra esse hash, sem necessidade de recuperar ou armazenar a senha original.

Além do hash de senha, o sistema utiliza códigos temporários para autenticação em dois fatores e recuperação de senha. Esses códigos possuem prazo de validade, o que reduz a possibilidade de reutilização indevida. Embora tais códigos não substituam a criptografia de transporte, eles compõem uma camada adicional de verificação no processo de autenticação.

Para implantação em ambiente real, recomenda-se o uso obrigatório de HTTPS, configuração segura de cookies de sessão, proteção contra CSRF e revisão periódica dos parâmetros de segurança aplicados ao hash de senhas.

6.6 - Identificação dos ativos do sistema

Ativo	Descrição	Tipo
Dados dos usuários	Nome, e-mail, telefone, CPF e	Dado pessoal

	endereço.	
Dados de dependentes	Nome, parentesco e data de nascimento.	Dado pessoal
Dados de pets	Nome, espécie e raça.	Informação complementar
Credenciais de usuários	Senhas armazenadas exclusivamente em formato de hash.	Credencial
Sessões autenticadas	Estado de autenticação mantido temporariamente pelo sistema.	Controle de acesso
Códigos 2FA e recuperação	Códigos temporários enviados por e-mail.	Autenticação
Consentimentos	Aceite, revogação, versão do termo, IP, User-Agent e datas.	Registro jurídico/técnico
Banco de dados	Repositório de informações persistentes do sistema.	Infraestrutura
Variáveis de ambiente	Credenciais técnicas não versionadas no repositório.	Credencial técnica
PDFs exportados	Documentos contendo dados pessoais do usuário.	Documento sensível
Funcionalidade de exclusão	Recurso de eliminação dos dados pessoais do titular.	Direito do titular

6.7 - Identificação de ameaças e vulnerabilidades

A documentação considera ameaças associadas à autenticação, ao tratamento de dados pessoais e ao uso da aplicação web. Como a equipe de desenvolvimento passou a tratar as correções apontadas como implementadas no ciclo de atualização, os itens abaixo são descritos como riscos controlados ou monitorados, e não como falhas impeditivas do protótipo.

Ameaça	Possível impacto	Controle associado
Força bruta	Tentativas sucessivas de adivinhação de senha.	Rate limiting e 2FA.
Vazamento de senha	Exposição de credenciais caso o banco seja acessado indevidamente.	Hash com Argon2.
SQL Injection	Manipulação maliciosa de consultas ao banco.	Consultas parametrizadas.
Acesso não autorizado	Uso de funcionalidades sem permissão adequada.	Sessão autenticada e recomendação de perfis de acesso.
Exposição de variáveis sensíveis	Vazamento de credenciais técnicas.	.env fora do versionamento por .gitignore.
Uso indevido de sessão	Acesso posterior em sessão abandonada.	Expiração de sessão e encerramento após exclusão de

		dados.
Tratamento sem consentimento	Tratamento de dados sem manifestação do titular.	Registro, revogação e novo aceite de consentimento.
Manutenção de dados além do necessário	Retenção indevida após solicitação do titular.	Funcionalidade de exclusão dos dados pessoais.

6.8 - Associação risco × contramedida

Risco identificado	Contramedida implementada ou documentada
Descoberta de senha por força bruta	Limitação de tentativas de login e autenticação em dois fatores.
Vazamento de senhas armazenadas	Hash com Argon2 no cadastro e na redefinição de senha.
Acesso indevido mesmo com senha válida	Envio e validação de código 2FA temporário.
SQL Injection	Uso de consultas parametrizadas no acesso ao banco.
Exposição de credenciais no repositório	Uso de variáveis de ambiente e exclusão do .env pelo .gitignore.
Sessões abertas por tempo excessivo	Expiração automática de sessão e encerramento após exclusão de dados.
Tratamento de dados sem consentimento	Registro de aceite dos termos, versão, IP, User-Agent e data.
Impossibilidade de retirada do consentimento	Revogação de consentimento e possibilidade de novo aceite.
Falta de transparência sobre dados armazenados	Exportação dos dados pessoais em PDF.
Retenção indevida de dados pessoais	Funcionalidade de exclusão dos dados do usuário.
Acesso administrativo inadequado	Adoção de controle de perfil e permissões conforme evolução do protótipo.
Requisições forjadas em formulários	Adoção de tokens CSRF como controle recomendado para ambiente de produção.

Com essas atualizações, a documentação técnico-científica evidencia um sistema com mecanismos de proteção de credenciais, autenticação reforçada, gestão de consentimento, exportação e exclusão de dados, aproximando o protótipo dos princípios de Segurança da Informação e proteção de dados pessoais.

6.9 - Registro de logs de tentativa de login

A versão atualizada do sistema passa a contemplar o registro estruturado das tentativas de autenticação na tabela logs_auth. Esse recurso amplia a rastreabilidade do processo de login e permite identificar tentativas bem-sucedidas e malsucedidas, associando-as ao e-mail informado, endereço IP, User-Agent, motivo de falha e data/hora do evento.

O registro de logs de autenticação complementa o mecanismo de rate limiting, pois não apenas limita tentativas sucessivas, mas também preserva evidências técnicas para auditoria, investigação de acessos suspeitos e análise de padrões de falha. Dessa forma, o sistema fortalece o controle de acesso e a capacidade de resposta a incidentes relacionados à autenticação.

Campo	Tipo	Finalidade no sistema
id	SERIAL PRIMARY KEY	Identificador único do evento de autenticação.
email	VARCHAR(255)	E-mail informado na tentativa de login, mesmo em caso de falha.
sucesso	BOOLEAN NOT NULL	Indica se a tentativa de autenticação foi concluída com sucesso.
ip	VARCHAR(45)	Endereço IP de origem, incluindo suporte a IPv4 e IPv6.
user_agent	TEXT	Informações do navegador ou cliente utilizado na tentativa.
motivo_falha	TEXT	Descrição da razão da falha, quando aplicável.
criado_em	TIMESTAMP DEFAULT CURRENT_TIMESTAMP	Data e hora do registro do evento.

Figura – Fluxo de registro de logs de autenticação no sistema SAECA.

Com essa estrutura, as tentativas de login passam a compor os ativos informacionais do sistema, uma vez que representam evidências de segurança e podem apoiar auditoria, detecção de força bruta, análise de acessos indevidos e revisão de incidentes.

6.10 - Atualização da matriz risco × contramedida para logs de autenticação

A inclusão da tabela logs_auth amplia a associação entre riscos de autenticação e contramedidas de segurança, pois permite documentar eventos relevantes para a rastreabilidade operacional do sistema. A tabela a seguir apresenta a relação entre os principais riscos e o uso dos logs como mecanismo de apoio à segurança.

Risco relacionado	Uso do log de autenticação	Benefício de segurança
Força bruta	Registro de múltiplas falhas por e-mail, IP ou User-Agent.	Permite identificar padrões de ataque e apoiar bloqueios ou revisões.
Credenciais inválidas	Armazenamento do motivo de falha associado à tentativa.	Facilita diagnóstico e auditoria sem expor a senha informada.
Acesso suspeito	Registro de IP e User-Agent em tentativas bem-sucedidas e malsucedidas.	Apoia investigação de acessos atípicos ou não reconhecidos.

Falha no segundo fator	Registro da tentativa sem sucesso após validação 2FA.	Permite diferenciar falha de senha e falha na etapa adicional de autenticação.
Incidente de segurança	Preservação de data e hora do evento.	Favorece rastreabilidade temporal e reconstrução dos eventos.

Por tratar-se de registro relacionado à segurança, recomenda-se que os logs de autenticação sejam protegidos contra alteração indevida, acessados apenas por perfis autorizados e mantidos por período compatível com a finalidade de auditoria e resposta a incidentes.

6.11 - Uso de artigos científicos e/ou normas técnicas

Além da análise direta do código-fonte, a documentação técnico-científica do sistema SAECA foi apoiada em referências legais, técnicas e normativas reconhecidas, com o objetivo de fortalecer a fundamentação acadêmica do projeto e relacionar as funcionalidades implementadas a boas práticas de Segurança da Informação, autenticação, proteção de credenciais e tratamento de dados pessoais.

A Lei Geral de Proteção de Dados Pessoais (LGPD) foi utilizada como principal referência legal para orientar os aspectos relacionados ao tratamento de dados pessoais, consentimento, transparência, revogação e exclusão de dados pelo titular. Essa base normativa sustenta a análise das funcionalidades de aceite de termos, registro de consentimento, exportação de dados em PDF e exclusão das informações pessoais do usuário.

No campo da segurança em aplicações web, foram consideradas recomendações da OWASP, especialmente o OWASP Top 10, documento de referência utilizado para conscientização sobre riscos críticos em aplicações web. Essa referência contribui para a análise de ameaças como falhas de controle de acesso, falhas criptográficas, injeção, autenticação inadequada e configurações inseguras, aspectos diretamente relacionados aos riscos tratados no SAECA.

Para fundamentar o processo de autenticação, foram consideradas as diretrizes do NIST Special Publication 800-63B-4, voltadas à autenticação digital e ao gerenciamento de autenticadores. Essa referência reforça a importância de controles adequados no processo de verificação de identidade, proteção de credenciais, autenticação multifator e gerenciamento do ciclo de vida dos autenticadores.

Quanto ao armazenamento seguro de senhas, foi considerada a especificação técnica RFC 9106, que descreve o Argon2 como função memory-hard voltada ao hash de senhas e aplicações relacionadas. Essa referência fundamenta a escolha do Argon2 no projeto, uma vez que o sistema utiliza esse mecanismo para evitar o armazenamento de senhas em texto puro e reduzir os impactos de eventual exposição da base de dados.

Dessa forma, o desenvolvimento e a documentação do sistema foram associados a fontes reconhecidas, permitindo demonstrar que os mecanismos implementados no protótipo estão

alinhados a boas práticas técnicas e a requisitos legais aplicáveis ao contexto de sistemas web que tratam dados pessoais.

6.12 - Referências normalizadas

As referências utilizadas na documentação foram organizadas em formato acadêmico, contemplando legislação brasileira, documentos técnicos internacionais e materiais de referência em segurança de aplicações web.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/113709.htm. Acesso em: 25 maio 2026.

BIRYUKOV, Alex; DWORKIN, Morris; KRAWCZYK, Hugo; NIKEGHBAL, Sahand. RFC 9106: Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications. Internet Research Task Force, 2021. Disponível em: <https://www.rfc-editor.org/rfc/rfc9106.html>. Acesso em: 25 maio 2026.

NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY. NIST Special Publication 800-63B-4: Digital Identity Guidelines: Authentication and Authenticator Management. Gaithersburg: NIST, 2025. Disponível em: <https://csrc.nist.gov/pubs/sp/800/63/b/4/final>. Acesso em: 25 maio 2026.

OPEN WORLDWIDE APPLICATION SECURITY PROJECT. OWASP Top 10:2021: The Ten Most Critical Web Application Security Risks. OWASP Foundation, 2021. Disponível em: <https://owasp.org/Top10/2021/>. Acesso em: 25 maio 2026.

OPEN WORLDWIDE APPLICATION SECURITY PROJECT. OWASP Top Ten Web Application Security Risks. OWASP Foundation. Disponível em: <https://owasp.org/www-project-top-ten/>. Acesso em: 25 maio 2026.

7.1 - Resumo entre 200 e 300 palavras

O presente projeto consiste no desenvolvimento de um protótipo de sistema web denominado SAECA, concebido para integrar funcionalidades de cadastro, autenticação, gerenciamento de informações e proteção de dados pessoais de usuários, dependentes e pets. A aplicação foi desenvolvida em Python, com o framework Flask, banco de dados Supabase e recursos complementares para envio de e-mails, geração de documentos, controle de sessão e autenticação. A solução contempla cadastro de usuários, autenticação com verificação em duas etapas, recuperação e redefinição de senha, gerenciamento de perfil, registro e revogação de consentimento, exportação das informações em PDF, exclusão dos dados pessoais pelo titular e registro de logs de tentativas de login. Entre os mecanismos de segurança, destacam-se o armazenamento de senhas por hash com Argon2, a utilização de variáveis de ambiente não versionadas para proteção de credenciais técnicas, consultas parametrizadas, limitação de

tentativas de autenticação, expiração de sessão e rastreabilidade de eventos de autenticação por meio da tabela logs_auth. O projeto também considera princípios da Lei Geral de Proteção de Dados Pessoais, especialmente quanto ao consentimento, transparência, segurança, revogação e eliminação de dados pessoais. Dessa forma, o protótipo evidencia a aplicação prática de medidas técnicas e organizacionais voltadas à mitigação de riscos relacionados ao acesso indevido, exposição de credenciais e tratamento inadequado de dados pessoais em sistemas web.

7.2 - Objetivo Claramente Definido

O objetivo geral do projeto é desenvolver um protótipo de sistema web seguro para cadastro, autenticação e gerenciamento de dados pessoais, aplicando mecanismos de Segurança da Informação e recursos associados à proteção de dados, ao controle de acesso e ao exercício de direitos do titular.

De forma específica, o projeto busca permitir o cadastro de usuários, dependentes e pets; autenticar usuários por senha e segundo fator; proteger senhas por hash; limitar e registrar tentativas de login; registrar, revogar e renovar consentimentos; exportar dados pessoais em PDF; e permitir a exclusão dos dados do titular, alinhando funcionalidades técnicas a princípios da LGPD.

7.3 - Metodologia Técnica Descrita

A metodologia adotada baseou-se no desenvolvimento incremental de uma aplicação web estruturada em camadas, separando interface, rotas, controladores, modelos e utilitários. O backend foi implementado em Python com Flask, enquanto o Supabase foi utilizado para persistência das informações. O acesso aos dados é realizado por consultas parametrizadas, reduzindo riscos de injeção SQL.

Para a proteção de credenciais, foi empregada a biblioteca Argon2, responsável pela geração e verificação de hashes de senha no cadastro, login e redefinição de senha. A autenticação foi complementada por verificação em duas etapas, com envio de código temporário por e-mail, limitação de tentativas e registro estruturado de eventos de autenticação na tabela logs_auth.

7.4 - Mecanismos de Segurança Apresentados

Os principais mecanismos de segurança apresentados são o armazenamento de senhas com Argon2, autenticação em dois fatores, limitação de tentativas de login, registro de logs de autenticação, expiração de sessão, consultas parametrizadas e uso de variáveis de ambiente para credenciais técnicas. Os logs registram e-mail informado, sucesso ou falha, IP, User-Agent, motivo de falha e data/hora, contribuindo para auditoria e rastreabilidade.

A exclusão dos dados pessoais reforça a proteção do titular e contribui para a minimização de retenção indevida. Após a operação, a sessão é encerrada, reduzindo a possibilidade de acesso posterior a partir de um estado autenticado já inválido.

7.5 - Conformidade com a LGPD Explicitada

O sistema apresenta aderência a princípios fundamentais da LGPD por tratar dados pessoais como nome, e-mail, telefone, CPF, endereço e informações associadas a dependentes. A conformidade é evidenciada pelo registro do consentimento durante o cadastro, pela possibilidade de revogação, pela exportação dos dados em PDF e pela exclusão dos dados pessoais mediante solicitação do usuário.

Essas funcionalidades dialogam com princípios como transparência, segurança, livre acesso, autodeterminação informativa, responsabilização e eliminação de dados. A documentação também reconhece a importância de políticas complementares de retenção, auditoria e controle de acesso aos logs, especialmente por se tratarem de registros associados à segurança operacional do sistema.

7.6 - Terminologia Técnica Adequada

A documentação utiliza terminologia técnica compatível com Segurança da Informação, desenvolvimento web e proteção de dados pessoais. São empregados conceitos como hash de senha, autenticação, autenticação em dois fatores, sessão, expiração de sessão, rate limiting, consultas parametrizadas, SQL Injection, variáveis de ambiente, consentimento, revogação, titular dos dados, eliminação de dados, LGPD, User-Agent e endereço IP.

Esses termos são aplicados de forma coerente com as funcionalidades implementadas e com os mecanismos de segurança descritos no sistema.

7.7- Qualidade Textual e Científica

A redação do resumo científico foi estruturada com linguagem formal, objetiva e adequada ao contexto acadêmico. O texto apresenta o problema abordado, a solução desenvolvida, as tecnologias utilizadas, os mecanismos de segurança incorporados e a relação do protótipo com a proteção de dados pessoais.

A organização textual segue uma sequência lógica, partindo da contextualização do sistema, passando pela metodologia técnica e finalizando com a análise dos mecanismos de segurança e conformidade. Dessa forma, a documentação demonstra coerência, precisão terminológica e fundamentação técnica adequadas a um projeto aplicado na área de Segurança da Informação.

CONSIDERAÇÕES FINAIS

O presente trabalho teve como objetivo documentar os requisitos de segurança da informação aplicados ao sistema SAECA, um protótipo de aplicação web desenvolvido em ambiente acadêmico para fins de cadastro, autenticação e gerenciamento de dados pessoais de usuários, dependentes e pets. A análise do sistema permitiu verificar que os mecanismos implementados

pela equipe demonstram aderência a boas práticas técnicas reconhecidas internacionalmente, ao mesmo tempo em que evidenciam o esforço de alinhamento às exigências da Lei Geral de Proteção de Dados Pessoais (LGPD — Lei nº 13.709/2018).

No que concerne à autenticação e à gestão de credenciais, destaca-se a adoção do algoritmo Argon2 para o armazenamento seguro de senhas, escolha fundamentada na RFC 9106 e reconhecida como solução de referência para proteção de credenciais em sistemas modernos. O uso de parâmetros de custo configuráveis e de salt automático reforça a resistência do sistema a ataques de força bruta e a ataques baseados em tabelas de hashes previamente conhecidas. A implementação de autenticação em dois fatores (2FA) por e-mail, com código numérico de seis dígitos e expiração de cinco minutos, acrescenta uma camada complementar de proteção ao processo de login, reduzindo o risco de acesso indevido mesmo diante do comprometimento das credenciais primárias.

A proteção contra ataques de força bruta foi contemplada por meio do mecanismo de rate limiting, implementado com a biblioteca Flask-Limiter. Foram estabelecidos limites globais de 2.000 requisições por dia e 200 por hora para as rotas gerais da aplicação, e limites mais restritivos para as rotas críticas de autenticação, admitindo no máximo cinco tentativas por minuto e vinte por hora. O controle de sessão foi estruturado com mecanismos de expiração e invalidação no logout, por meio do comando `session.clear()`, reduzindo o risco de permanência indevida de sessões ativas. A criação da tabela `logs_auth` para o registro estruturado de tentativas de autenticação — contemplando campos como endereço IP, User-Agent, resultado da tentativa, motivo de falha e data do evento — amplia a rastreabilidade operacional do sistema e constitui um ativo informacional relevante para auditoria e resposta a incidentes.

Em relação à conformidade com a LGPD, o sistema demonstra aderência a princípios fundamentais da lei, especialmente os de finalidade, necessidade, transparência, segurança e livre acesso. O registro explícito de consentimento durante o cadastro, com armazenamento de data, hora, endereço IP e versão do termo, viabiliza a comprovação documental do aceite. As funcionalidades de revogação do consentimento, exportação dos dados pessoais em formato PDF e exclusão dos registros associados ao titular atendem a direitos assegurados pela legislação vigente. O encerramento automático da sessão após a exclusão dos dados reforça a consistência entre o estado do sistema e a situação jurídica do usuário.

A documentação reconhece, com transparência técnica, limitações que devem ser endereçadas em versões futuras do sistema. A ausência de um mecanismo centralizado de verificação de sessão nas rotas internas — como um decorator do tipo `login_required` — representa uma lacuna no controle de acesso que pode, em determinados cenários, permitir a navegação sem autenticação prévia. De modo similar, a implementação de TLS/HTTPS e de tokens de proteção contra CSRF foi indicada apenas como recomendação para ambientes de produção, não estando presente na versão do protótipo documentada. O controle de acesso diferenciado por perfis de usuário também foi apontado como uma evolução necessária, ainda pendente de implementação.

A fundamentação técnico-científica do projeto apoiou-se em referências normativas reconhecidas, incluindo a LGPD, o OWASP Top 10, o NIST Special Publication 800-63B-4 e a RFC 9106, o que confere sustentação acadêmica às escolhas de implementação e demonstra que as decisões técnicas adotadas não foram arbitrárias, mas derivadas de padrões e boas práticas consolidados na área de segurança da informação.

Conclui-se que o sistema SAECA constitui uma base técnica sólida para o desenvolvimento de aplicações web seguras, integrando de forma coerente mecanismos de proteção de credenciais, controle de autenticação, mitigação de ataques e conformidade com a legislação de proteção de dados. As limitações identificadas, por estarem explicitamente documentadas e associadas a propostas de evolução, não comprometem o valor técnico e acadêmico do trabalho, mas apontam o caminho necessário para a maturação do protótipo em direção a um sistema apto à implantação em ambiente de produção.